

TP1 — Continual Learning en Seq-CIFAR-10

I309 Visión Artificial Avanzada · Universidad de San Andrés

María Trinidad Pujol Sierra · Benjamín Vitale

1 Introducción

El aprendizaje continuo (*continual learning*) aborda el problema de entrenar un modelo secuencialmente sobre múltiples tareas sin perder el conocimiento adquirido en las anteriores, fenómeno conocido como **olvido catastrófico** (*catastrophic forgetting*).

En este trabajo implementamos y comparamos cuatro métodos de aprendizaje continuo sobre **Seq-CIFAR-10**: CIFAR-10 dividido en 5 tareas secuenciales de 2 clases cada una. Los métodos evaluados son Naive Fine-Tuning (baseline), EWC, LwF y Co²L, bajo dos escenarios de evaluación: **Class-IL** (sin pista de tarea en inferencia) y **Task-IL** (con pista de tarea).

2 Etapa 1 — Dataset: Seq-CIFAR-10

2.1 Partición de tareas

CIFAR-10 se divide en 5 tareas disjuntas de 2 clases cada una:

Tarea	Clases	Train	Test
0	airplane, automobile	10 000	2 000
1	bird, cat	10 000	2 000
2	deer, dog	10 000	2 000
3	frog, horse	10 000	2 000
4	ship, truck	10 000	2 000

Table 1: Partición de Seq-CIFAR-10 en 5 tareas secuenciales.

2.2 Decisiones de diseño

Descarga automática del dataset. Se utiliza `torchvision.datasets.CIFAR10(download=True)`, que descarga y verifica la integridad del dataset en el primer uso.

Almacenamiento lazy de imágenes. Las imágenes se mantienen en formato PIL y los transforms se aplican en `__getitem__`, lo que permite usar distintos pipelines de augmentación (estándar o SupCon) sobre los mismos datos sin duplicar memoria.

Pre-cómputo de índices por tarea. Al construir SeqCIFAR10, se calculan los índices de cada tarea una sola vez. Construir un `DataLoader` para cualquier tarea es luego $O(1)$, sin necesidad de volver a filtrar el dataset completo.

Augmentaciones de entrenamiento:

- *Estándar*: `RandomCrop(32, padding=4) + RandomHorizontalFlip + normalización`.
- *SupCon* (`TwoViewTransform`): dos vistas independientes por imagen con `RandomResizedCrop(32, scale=(0.2, 1.0)) + ColorJitter + RandomGrayscale`. El `DataLoader` produce batches de forma $(B, 2, C, H, W)$.

2.3 Replay Buffer

Se implementó un `ReplayBuffer` de capacidad fija usando **reservoir sampling** (Vitter, 1985). Garantiza que en cualquier momento el buffer es una muestra uniforme de todas las imágenes vistas

hasta ahora, sin necesidad de balanceo manual entre tareas: cada nueva muestra desplaza una posición aleatoria del buffer con probabilidad `capacity / n_seen`.

3 Etapa 2 — Pre-entrenamiento con SupCon

3.1 Arquitectura del backbone

Se usa **ResNet-18 adaptado para imágenes de 32×32** con dos modificaciones clave respecto al ResNet-18 estándar:

- **conv1**: 7×7 stride-2 $\rightarrow 3 \times 3$ **stride-1** (sin reducción de resolución espacial en la primera capa).
- **maxpool**: eliminado (`nn.Identity`), evitando el downsampling excesivo en imágenes pequeñas.

Sobre el encoder (512-dim) se montan dos cabezas:

Cabeza	Arquitectura	Uso
Projection head	Linear(512,512) $\rightarrow$ ReLU $\rightarrow$ Linear(512,128) $\rightarrow$ L2-norm	SupConLoss
Classifier	Linear(512, 10)	CrossEntropyLoss

Table 2: Cabezas montadas sobre el encoder ResNet-18.

3.2 Supervised Contrastive Loss (SupCon)

La loss de Khosla et al. (NeurIPS 2020) trata a todas las vistas de imágenes de la misma clase como positivos:

$$\ell_i = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{a \neq i} \exp(z_i \cdot z_a / \tau)} \quad (1)$$

Se implementó con el truco log-sum-exp para estabilidad numérica. La clase acepta un `anchor_mask` booleano para que, en Co²L, las muestras del buffer actúen solo como negativos (no como anchors).

3.3 Pre-entrenamiento en Tarea 0

Hiperparámetro	Valor
Epochs	200
Optimizer	SGD
Learning rate	0.5
Temperatura τ	0.07
Batch size	128

Table 3: Hiperparámetros del pre-entrenamiento SupCon.

El pre-entrenamiento se realiza únicamente sobre la Tarea 0 (airplane / automobile), usando el loader SupCon de dos vistas. Se guardaron snapshots del backbone en las épocas 0, 100 y 200 para visualizar la evolución de las representaciones con t-SNE.

Decisión de diseño: pre-entrenar solo sobre Task 0 busca inicializar el encoder con representaciones contrastivas antes de iniciar el escenario continuo, en línea con la motivación de Co²L de que un buen espacio de representaciones es más transferible que uno entrenado con entropía cruzada.

3.4 Linear Probe

Después del pre-entrenamiento se congela el encoder y se entrena solo el clasificador lineal sobre Tarea 0 (100 epochs, SGD, lr = 1.0). El encoder permanece *frozen* durante la probe y se descongela

antes de iniciar la fase de aprendizaje continuo.

Resultado: test accuracy en Task 0 = **99.4 %**, confirmando que las representaciones SupCon son altamente linealmente separables para las clases airplane / automobile.

4 Etapa 3 — Métodos de Aprendizaje Continuo

Todos los métodos parten del **mismo backbone pre-entrenado** (encoder descongelado) y siguen la misma interfaz `begin_task` → `train_task` → `end_task`. Salvo Co²L, entrenan 50 épocas por tarea con SGD (lr = 0.01, momentum = 0.9, weight_decay = 5e-4).

4.1 Naive Fine-Tuning (baseline)

Re-entrena el modelo con entropía cruzada estándar en cada tarea sin ningún mecanismo anti-olvido. Sirve como cota inferior para el forgetting.

$$L = L_{\text{CE}}(\text{tarea actual}) \quad (2)$$

4.2 EWC — Elastic Weight Consolidation

Después de completar cada tarea, estima la **diagonal de la Fisher Information Matrix** como proxy de importancia de cada parámetro:

$$F_i^{(t)} = \mathbb{E} \left[\left(\frac{\partial \log p(y|x, \theta)}{\partial \theta_i} \right)^2 \right] \approx \frac{1}{N} \sum_n \left(g_i^{(n)} \right)^2 \quad (3)$$

Durante el entrenamiento de la tarea siguiente, penaliza cambios en parámetros importantes:

$$L = L_{\text{CE}} + \frac{\lambda}{2} \sum_{t' < t} \sum_i F_i^{(t')} \cdot \left(\theta_i - \theta_i^{*(t')} \right)^2 \quad (4)$$

Se usó $\lambda = 400$, seleccionado por ser el valor que mejor equilibró estabilidad y plasticidad. La Fisher se acumula para todas las tareas pasadas.

4.3 LwF — Learning without Forgetting

Antes de entrenar cada tarea ($t > 0$), se guarda un **snapshot frozen del modelo** como teacher. Durante el entrenamiento, se añade una pérdida de destilación:

$$L = L_{\text{CE}}(\text{nueva tarea}) + \alpha \cdot T^2 \cdot \text{KL} \left(\text{softmax} \left(\frac{z_{\text{teacher}}}{T} \right) \parallel \log \text{softmax} \left(\frac{z_{\text{student}}}{T} \right) \right) \quad (5)$$

El factor T^2 reescala el gradiente de los soft targets a la misma magnitud que los hard targets (Hinton et al., 2015). La destilación se realiza sobre los **10 logits completos**. Se usaron $\alpha = 1.0$ y $T = 2.0$.

4.4 Co²L — Contrastive Continual Learning

Implementación de Cha et al. (ICCV 2021). Combina aprendizaje contrastivo supervisado con destilación asimétrica. Cada tarea se entrena en **dos fases**:

Fase 1 — Representaciones (100 epochs, encoder + projector):

$$L = L_{\text{SupCon}}(\text{batch conjunto}) + \lambda \cdot L_{\text{A-Distill}} \quad (6)$$

L_{SupCon} se calcula sobre un batch conjunto de la tarea actual (como anchors, dos vistas) y muestras del replay buffer (solo como negativos de una vista). $L_{\text{A-Distill}}$ (*Asymmetric Distillation Loss*) alinea las features del encoder actual con las del teacher frozen:

$$\ell_i = -\log \frac{\exp(z_i^{\text{curr}} \cdot z_i^{\text{prev}} / \tau)}{\sum_j \exp(z_i^{\text{curr}} \cdot z_j^{\text{prev}} / \tau)} \quad (7)$$

Fase 2 — Clasificador (50 epochs, encoder frozen): entropía cruzada estándar sobre datos de la tarea actual + samples del replay buffer. El clasificador se reinicializa con Kaiming-uniform antes de esta fase.

Hiperparámetro	Valor
Temperatura τ	0.07
λ (co2l_lambda)	1.0
LR fase 1	0.03 (SGD, cosine annealing)
LR fase 2	1.0 (SGD, cosine annealing)
Epochs fase 1 / fase 2	100 / 50 por tarea
Buffer size	500 (reservoir sampling)
Samples del buffer por batch	32

Table 4: Hiperparámetros de Co²L.

5 Etapa 4 — Comparación de Resultados

5.1 Tabla resumen

Método	Class-IL final	Task-IL final	Avg Forg. (C)	Avg Forg. (T)
Naive	0.196	0.648	0.970	0.405
EWC	0.196	0.637	0.967	0.415
LwF	0.229	0.960	0.488	0.010
Co ² L	0.351	0.836	0.729	0.160

Table 5: Métricas finales de los cuatro métodos sobre Seq-CIFAR-10.

5.2 Accuracy por tarea al finalizar las 5 tareas

Class-IL (sin pista de tarea)						Task-IL (con pista de tarea)					
Método	T0	T1	T2	T3	T4	Método	T0	T1	T2	T3	T4
Naive	0.000	0.000	0.000	0.000	0.978	Naive	0.823	0.427	0.459	0.551	0.978
EWC	0.000	0.000	0.000	0.000	0.979	EWC	0.500	0.596	0.568	0.544	0.979
LwF	0.057	0.002	0.001	0.115	0.974	LwF	0.983	0.904	0.953	0.982	0.979
Co ² L	0.255	0.129	0.139	0.292	0.938	Co ² L	0.946	0.720	0.750	0.788	0.977

Table 6: Accuracy por tarea al finalizar el entrenamiento secuencial.

5.3 Análisis

Naive y EWC son prácticamente equivalentes en Class-IL. Ambos alcanzan accuracy ~ 0.196 , que corresponde exactamente a clasificar solo la última tarea vista (Task 4 ≈ 0.978 , con un prior de 2/10 clases ≈ 0.20). EWC no logra mejorar sobre Naive en este escenario, posiblemente porque la penalización de Fisher (que preserva parámetros importantes en el espacio de pesos) no es suficiente para evitar el olvido de los logits de clases pasadas cuando el clasificador comparte todos los outputs.

LwF es el mejor método en Task-IL, con 96 % de accuracy promedio y un forgetting de apenas 1 %. La destilación sobre los logits del teacher funciona muy bien cuando se provee la pista de tarea, ya que el modelo puede consultar las cabezas correctas. En Class-IL sube a 22.9 %, pero sigue siendo bajo: sin pista de tarea, el modelo tiende a predecir clases de tareas recientes.

Co²L es el mejor en Class-IL (35.1 %), con una ganancia significativa de ~ 15 puntos porcentuales sobre LwF. El aprendizaje contrastivo fuerza representaciones más separables en el espacio de features, lo que ayuda cuando el modelo debe discriminar entre las 10 clases simultáneamente. Sin embargo, su forgetting en Class-IL (72.9 %) es mayor que el de LwF (48.8 %), y en Task-IL queda por debajo de LwF.

Brecha Class-IL / Task-IL. Todos los métodos presentan una brecha significativa entre Task-IL y Class-IL (especialmente Naive y EWC, con ~ 45 puntos). Esto refleja el problema fundamental de Class-IL: el clasificador lineal comparte los 10 logits entre todas las tareas, y sin pista de tarea el modelo tiende a favorecer las clases más recientemente entrenadas.

6 Figuras generadas

Figura	Descripción
supcon_pretrain_loss.png	Curva de loss SupCon durante el pre-entrenamiento (200 epochs)
supcon_embeddings_stages.png	t-SNE de features en épocas init / mid / final
linear_probe_curves.png	Curvas de loss y accuracy del linear probe en Task 0
class_il_accuracy.png	Accuracy promedio Class-IL vs tareas (4 métodos)
task_il_accuracy.png	Accuracy promedio Task-IL vs tareas (4 métodos)
forgetting.png	Curvas de forgetting por tarea (Class-IL)
comparison.png	Panel comparativo Class-IL y Task-IL
heatmap_class_il.png	Heatmap de accuracy por tarea (Class-IL)
heatmap_task_il.png	Heatmap de accuracy por tarea (Task-IL)

Table 7: Listado de figuras generadas durante los experimentos.

7 Conclusiones

- **LwF** es el mejor método en Task-IL: la destilación de logits es suficiente para preservar el conocimiento de tareas pasadas cuando se conoce la tarea en inferencia.
- **Co²L** es el mejor en Class-IL: las representaciones contrastivas mejoran la separabilidad entre todas las clases, lo que es crítico cuando no hay pista de tarea.
- **EWC** no mejora sobre Naive en ninguno de los dos escenarios con la configuración utilizada. La regularización de Fisher sobre los pesos del backbone no se traduce en preservación de los logits cuando el clasificador está compartido.
- El **gap Class-IL / Task-IL** es el resultado más llamativo: refleja que el problema de Class-IL es fundamentalmente más difícil y que ninguno de los métodos implementados lo resuelve completamente.

Referencias

- [1] Kirkpatrick et al., *Overcoming catastrophic forgetting in neural networks*, PNAS 2017.
- [2] Li & Hoiem, *Learning without Forgetting*, TPAMI 2018.
- [3] Khosla et al., *Supervised Contrastive Learning*, NeurIPS 2020.
- [4] Cha et al., *Co²L: Contrastive Continual Learning*, ICCV 2021.
- [5] Vitter, J.S., *Random sampling with a reservoir*, TODS 1985.
- [6] Hinton et al., *Distilling the knowledge in a neural network*, NIPS Workshop 2015.